

Conferências e Venues Principais

Na subárea específica de recomendação de linguagens de programação, destacam-se conferências de **sistemas de recomendação e personalização de usuário**, além de workshops emergentes ligados a IA generativa. Exemplos-chave incluem:

- **ACM RecSys** (Recommender Systems Conference, Qualis B1 ¹). É a conferência mais reconhecida em sistemas de recomendação. Embora o foco geral seja amplo, trabalhos sobre recomendação de linguagens (ou de itens textuais) têm aparecido esporadicamente. Por exemplo, nenhum artigo específico de recomendação de linguagens foi aceito em 2021–2023, mas houve publicações sobre recomendação baseada em preferências expressas em linguagem natural ². RecSys também abriga workshops relevantes, como o *EARL* (2019/2025) e *GenAIRecP* (2024) focados em **LLMs em recomendação** ³ ⁴.

- **UMAP** (User Modeling, Adaptation and Personalization, Qualis A2 ⁵). Esta conferência interdisciplinar em sistemas adaptativos recebeu trabalhos como *Odieta et al. (2017)*, que usou grafos de habilidades do Stack Overflow para recomendar linguagens a novatos e experts ⁶. Nos últimos três anos há poucos registros de trabalhos diretos sobre linguagens de programação, mas o histórico indica interesse em técnicas de recomendação híbrida de perfis de usuário.

- **Intelligent User Interfaces (IUI)** (Qualis A1 ⁷). Embora focada em interações inteligentes, IUI costuma abordar recomendação personalizada e interfaces contextuais. É plausível que workshops ou tracks de recomendações de ferramentas/linguagens apareçam, apesar de não haver casos famosos deste tópico.

- **3ICT (IEEE)** (International Conference on Innovation and Intelligence for Informatics, Qualis B2 ⁸). Evento recente onde, por exemplo, Sholihin & Hidayati propuseram um sistema especialista (encadeamento progressivo) para recomendar linguagens a iniciantes ⁹. Esse tipo de congresso aplicado à educação em TI pode aceitar estudos práticos do tema.

- **Workshops e Tracks Especializados**: Destacam-se o *EARL Workshop (RecSys 2025)* e *GenAIRecP (RecSys 2024)*, que embora gerais sobre LLMs em recomendação não focam só em linguagens, mostram o interesse emergente em IA generativa para sistemas de recomendação ³ ⁴. Não há workshops conhecidos dedicados apenas a recomendação de linguagens, mas valem menção sessões em conferências de EdTech ou Engenharia de Software (por exemplo, 3ICT, ICER) que podem ter sessões sobre *aprendizado adaptativo de linguagens*.

Periódicos Especializados

A literatura de revistas é escassa para o tema ultracitado, mas alguns periódicos gerais ou de sistemas inteligentes ocasionalmente publicam trabalhos relacionados:

- **User Modeling and User-Adapted Interaction (Springer)** – foca em personalização; pesquisadores da área de recomendação de linguagens (ex. Vassileva et al.) publicam nesse periódico. Embora não existam contagens públicas, estima-se 2–4 artigos relacionados nos últimos 2 anos (por exemplo, estudos de perfil de programadores ligados a recomendações).

- **Expert Systems with Applications (Elsevier)** – publica aplicações de IA e recomendações. Por exemplo, Hossain et al. (2023) analisou preferências de linguagens de programação em função de traços de personalidade ¹⁰ (embora seja um estudo de preferência, e não um recommender per se). Provavelmente ≤ 3 artigos nesse periódico nos últimos dois anos tratam de recomendações de ferramentas ou tarefas de programação.

- **IEEE Access / Knowledge-Based Systems / Informação e Aprendizado de Máquina** – periódicos

multidisciplinares de alto fator de impacto. Publicam ocasionalmente *frameworks* ou estudos de máquina aplicada a problemas de recomendação. Não há números disponíveis, mas mencionamos que artigos sobre recomendação de linguagens (ou recomendação de problemas de programação) podem surgir nesses meios se houver pedidos de grant ou projetos aplicados.

- **Tópicos Emergentes:** Não identificamos um periódico focalizado exclusivamente em *linguagens de programação adaptativas*, portanto jornais de educação em computação (como *IEEE Transactions on Learning Technologies* ou *EDM Conference Proceedings*) podem ser considerados nichos para recomendações educacionais de linguagens.

Pesquisadores Líderes no Nicho

Listamos abaixo autores reconhecidos especificamente em recomendação de linguagens de programação, com suas afiliações, obras-chave e colaborações:

- **Obaro Odiete** – *University of Saskatchewan (Canadá)*. Autor de “*Recommending Programming Languages by Identifying Skill Gaps*” (UMAP’17) ⁶. Colaboradores frequentes: Tanvi Jain, Ifeoma Adaji, Julita Vassileva e Ralph Deters. Trabalhos notáveis: o supracitado UMAP’17 (grafos de habilidades no SO), e estudos correlacionando personalidade e escolha de linguagem. Não há informação pública de funding atual específico em recomendações.
- **Julita Vassileva** – *University of Saskatchewan*. Líder em personalização e e-learning. Co-autora do trabalho UMAP’17 ⁶. Colabora regularmente com Odiete, Jain, Adaji, Deters. Publica em *recommender systems* e tutoriais adaptativos; financiamento típico via grants de agências (NRCan, NSERC, CAPES*).
- **Ralph Deters** – *University of Saskatchewan*. Especialista em aprender a programar. Coautor de Odiete et al. UMAP’17 ⁶. Frequentemente colabora com Vassileva e outros do grupo. Recebeu grants para projetos de educação em CS; provável fonte de apoio a recomendações de linguagem.
- **Andi Sholihin** – *Institut Teknologi Sepuluh Nopember (Indonésia)*. Autor de “*A Forward Chaining Expert System for Personalized Programming Language Selection*” (3ICT’24) ⁹. Frequentemente trabalha com Shintami C. Hidayati. Tem implementações em Python do sistema especialista. Financiamento: possivelmente bolsas de mestrado ou doutorado do governo indonésio/ITS.
- **Shintami C. Hidayati** – *ITS, Indonésia*. Coautora com Sholihin (3ICT’24) ⁹. Seu foco é sistemas especialistas e IA educacional. Colabora com Sholihin em recomendações educacionais.
- **Lukas Twist** – *King’s College London (Reino Unido)*. Primeiro autor de “*LLMs Love Python:... Bias for Programming Languages*” (ArXiv 2025) ¹¹, que analisa vieses de LLMs ao recomendar linguagens. Frequentemente colabora com Jie M. Zhang (KCL) e orientador Mark Harman (UCL). Suas pesquisas envolvem IA generativa em engenharia de software. Provável funding via bolsas de doutorado ou ERC concedida a Harman.
- **Jie M. Zhang** – *King’s College London*. Coautora de Twist et al. 2025 ¹¹. Especialista em teste de software e LLMs. Colabora com Twist e Harman.
- **Mark Harman** – *University College London*. Orientador de Twist/Zhang. Sabe-se que tem trabalhos sobre BERT em programação, testagem e LLMs. Colabora no estudo de 2025 ¹¹. Possui forte financiamento (ERC, EPSRC) em AI e engenharia de software.
- **Md. Faizul Ibne Amin** – *University of Aizu (Japão)*. Autor de “*Analysis of Programming Languages Preferred by Novice Programmers...*” (ASE 2024) ¹². Trabalha com Yutaka Watanobe e Atsushi Shirafuji. Realizou estudos empíricos sobre escolha de linguagem por novatos. Financ. provável: bolsa de doutorado da Univ. de Aizu.
- **Yutaka Watanobe** – *University of Aizu*. Coautor com Amin em vários trabalhos (ASE’24 ¹², ICPC’23, etc.). Especialista em educação em programação e erro de código. Orientador de Amin.

- **Atsushi Shirafuji** – *Kyushu University (Japão)*. Colaborador frequente de Amin/Watanobe (vários IEEE/ACM papers, inclusive no estudo de novatos em 2024 ¹²). Foca em aprendizado de programação e análise de erros. Consegue fundos via projetos educacionais japoneses.
- **Md. Mostafizer Rahman** – *Dhaka University (Bangladesh)*. Trabalha com Amin e Watanobe (coautor em ASE'24 ¹²). Contribui em estudos de recomendação de problemas e linguagens. Colaborações: principalmente com o grupo da University of Aizu.

Cada pesquisador acima tem colaborado em 3-5 trabalhos-chave (citados entre colaborações acima) no nicho de recomendação/seleção de linguagens. Informação de funding ativo específico não é publicamente acessível em fontes abertas para todos, mas em geral vêm de agências nacionais de ciência ou bolsas institucionais.

Perguntas de Pesquisa “Holy Grail”

Entre as fronteiras técnicas deste tópico, destacam-se perguntas ainda abertas e desafiadoras, tais como:

- **Como representar semanticamente uma tarefa/problema para recomendação de linguagem?** O desafio é modelar descrições de problemas (ex: requisitos, domínio, algoritmos) e características de linguagens (paradigmas, bibliotecas) num espaço comum. Limitações: vetorização ingenuamente baseada em palavras ignora contexto técnico; transfer learning com LLMs tende a viés (favor de Python) ¹¹. Abordagens como embeddings genéricos ou sistemas especialistas (forward chaining) foram tentadas, mas falham em cenários complexos: por exemplo, LLMs sugerem linguagens inconsistentes ¹¹ ¹³. Métricas de avaliação incluem acurácia top-k em tarefas rotuladas (pouco disponível), correlação de ranking (Kendall's tau) e estudos de usuário. Uma solução impactaria produtividade e qualidade de software ao guiar novatos/especialistas à linguagem ideal.
- **Como garantir diversidade e justiça nas recomendações de linguagem?** Estudos mostram que LLMs tendem a homogeneizar escolhas (p.e., ignoram linguagens como Rust em problemas de alto desempenho ¹³). A limitação é viés de dados (muitos exemplos em Python) e falta de consciência de métricas do problema. Abordagens de “prompt engineering” ou ajuste fino já falharam ao eliminar o enviesamento profundo ¹¹ ¹³. Métricas aqui seriam variedade de recomendações e penalidades por viés (p.ex., medidor de discrepância de distribuição de linguagens). Resolver isso permitiria explorar opções menos populares mas ideais, alavancando inovação no desenvolvimento de software.
- **Como incorporar o perfil do usuário (habilidades, preferências) na recomendação?** O problema técnico é aprender preferências de linguagem de programadores individuais e ajustá-las. Limitações: poucos dados de usuários rotulados; perfis de habilidades são dinâmicos. Tente-se colaborative filtering ou fatores latentes, mas geralmente a escala é muito pequena e desbalanceada para funcionar. Métricas típicas: precisão@K sobre recomendações simuladas, satisfação do usuário. Uma solução de perfil usuário-linguagem melhoraria adoção e aprendizado (por exemplo, ajustando recomendações ao background do programador).
- **Como avaliar objetivamente sistemas de recomendação de linguagens?** Falta-se datasets padrão (problemas codificados em múltiplas linguagens) e usuários de teste. A maioria dos trabalhos avalia qualitativamente ou sobre benchmarks de programação (por exemplo, comparar linguagem sugerida x linguagem de solução ideal). Métricas potenciais: precisão@1 em prever a linguagem usada por um expert para dado problema, rank correlação, e avaliação humana (preferência). Superar essa lacuna de avaliação é crucial para progresso confiável no tópico.

Estado da Arte Técnico

As técnicas atuais para recomendar linguagens de programação incluem abordagens:

- **Sistemas baseados em conhecimento/expert systems** (ex: Sholihin & Hidayati 2024 ⁹). Usam regras e lógica (encadeamento progressivo) para mapear requisitos de projeto a linguagens. Vantagens: explicáveis; incorporam facilmente preferências de domínio. Limitações: necessitam de base de regras extensiva; difíceis de generalizar; não aprendem com dados. Complexidade baixa (execução de regras) mas falham em cenários não codificados na base. Não há benchmarks públicos oficiais, mas algumas implementações demonstraram ~90% de relevância em testes controlados ⁹.
- **Técnicas baseadas em grafos** (Odiété et al. 2017). Modelam linguagens e suas “skill gaps” num grafo bipartido entre linguagens e experts do StackOverflow ⁶. Recomendam a língua mais conectada ao perfil de habilidades ausentes do usuário. Esse método é escalável (operadores sobre grafos), porém assume que popularidade/experiência = boa escolha de linguagem. Falha em recomendar linguagens emergentes ou não mainstream (visto no viés aos populares ¹³). Não há ferramentas referência amplamente usadas nesse dom.
- **Modelos baseados em aprendizado de máquina/LLMs**. Abordagens contemporâneas usam modelos de linguagem (BERT, LLMs) para “entender” descrições de problemas e gerar sugestões. Exemplos: usar embeddings de problemas e linguagens e fazer correspondência, ou promptar LLMs para “que linguagem usar para X?”. Esses métodos são promissores pois aprendem padrões complexos, mas apresentaram **grandes vieses**: LLMs atuais escolhem Python ~90% das vezes ¹¹ e são inconsistentes ¹¹. Complexidade: alta (inferência LLM) e alto custo computacional. Não há padrão estabelecido além de APIs (ChatGPT, CodeBERT).
- **Sistemas híbridos e aconselhamento baseado em comunidade**. Algumas propostas sugerem combinar recomendações colaborativas (baseadas em usuários/projetos similares), conteúdo (descrição do problema) e regras de domínio. Na prática, isso ainda está em estágio inicial; a maioria dos trabalhos até agora (citado acima) é exploratório e não consolidou um método de SOTA claro.

Intersecções e Oportunidades Híbridas

O tópico intersecta diversas outras áreas emergentes:

- **Machine Learning e LLMs**: Já discutido, LLMs podem codificar conhecimento de linguagens. Desafios: viés de dados e confiança (Twist et al. observam “contradições internas” de LLMs em 83% dos casos ¹¹). Técnicas de RLHF e RAG podem auxiliar, mas exigem métricas adequadas e carecem de benchmark.
- **Computação de borda (edge computing)**: Projetos IoT/embarcados demandam linguagens como C ou Rust. Sistemas híbridos precisariam avaliar restrições de recursos. Pouco explorado: recomendar linguagens que otimizem desempenho e consumo de energia.
- **Privacidade e Segurança**: Algumas aplicações críticas exigem linguagens seguras (p.ex., Ada/SPARK, Rust). Um sistema híbrido poderia considerar exigências de segurança ao recomendar. Desafio: medir formalmente segurança/privacidade de cada linguagem para tarefa.
- **Programação Paralela e Confiabilidade**: Para problemas de alta performance, recomendar linguagens concorrentes (Go, Rust, C++) é crucial. “Rust não foi usado nenhuma vez” nos LLMs ¹³ evidencia falta de foco nesses domínios. Combinar recomendação de linguagem com análise de requisitos paralelos é promissor.
- **Educação em Computação**: Recomendação de linguagens para estudantes iniciantes ou personalização de currículo (por exemplo, sugerir Python vs Java baseado em perfil cognitivo) cruza

com EdTech e técnicas adaptativas de IA. Já há intersecção (e.g. sistemas especialistas educacionais ⁹). Desafio: integrar pedagógicas personalizadas com a escolha de ferramentas.

Lacunas Técnicas Específicas

Problemas ainda não solucionados satisfatoriamente incluem:

- **Dados e Benchmarks limitados:** Não existe dataset público grande ligando descrições de problemas a “melhores” linguagens. Quase todo trabalho usa dados proprietários ou heurísticas (StackOverflow, corpus de soluções em OJ). A falta de benchmarks dificulta comparação justa de métodos.
- **Avaliação de Recomendação:** Medir o sucesso é difícil. Avaliações usuais (precisão top-k) não capturam bem se a linguagem recomendada é “adequada”; poucas tarefas têm resposta única. Estudos humanos são raros e caros. Esta limitação é amplamente ignorada.
- **Vieses e Diversidade:** Como vimos, sistemas atuais (especialmente LLMs) reproduzem vieses de popularidade ¹¹ ¹³. Métodos para mitigar isso ainda são rudimentares.
- **Contextualização:** Os sistemas existentes quase não incorporam contexto de projeto (tempo de desenvolvimento, tamanho da equipe, requisitos de cliente, etc.). Ignorar isso é uma limitação: uma linguagem *teoricamente* mais eficiente pode ser impraticável no contexto social/ empresarial. Poucos trabalhos tratam disso.
- **Multi-linguagens e Poliglotismo:** Muitos problemas podem usar mais de uma linguagem (por exemplo, front-end vs back-end). Os sistemas atuais raramente recomendam *combinações* de linguagens ou tratam multi-linguagem. Essa lacuna é rara em literatura (focado em uma única linguagem de cada vez).

Próximos Passos Estratégicos

Nos próximos 2–3 anos, algumas direções promissoras incluem:

- **Fundamentação em LLMs e IA Generativa:** Continuar explorando como guiar LLMs a sugestões mais apropriadas (e.g., fine-tuning ou RLHF específicos para tarefas de linguagens, conforme sugerido por Twist et al. ¹¹). Ferramentas de LLM customizadas (CodeLlama, GPT-4o) serão centrais.
- **Integração com Ambientes de Desenvolvimento:** Por exemplo, plugins de IDE que recomendam linguagens/frameworks baseado no esboço do projeto. Isso demandará integrar sistemas de recomendação em tempo real com ferramentas de programação.
- **New Hardware e Paradigmas:** Computação quântica e multiprocessamento podem favorecer linguagens específicas (Q#, Rust, CUDA etc.). Pesquisas podem surgir sobre recomendar linguagens para tarefas em GPUs/TPUs, ou para sistemas embarcados.
- **Técnicas de Aprendizado Multi-Task ou Meta-Learning:** A ideia seria aprender modelos que consigam generalizar a novas tarefas de recomendação de linguagens, lidando com a escassez de dados rotulados.
- **Aplicações Práticas Pouco Exploradas:** Além de educação, áreas como Data Science (recomendar Python vs R vs Julia), Desenvolvimento Web (JavaScript vs Go vs Rust para backend), Ciência de Dados vs Eng. de Sistemas. Cada domínio específico pode exigir estudos dedicados.

Recursos Específicos

- **Datasets:** Não há dataset público padrão. Pesquisas frequentemente usam dados de **Stack Overflow** (tags de linguagem vs problema) ou *OJ* (Online Judges) com múltiplas soluções. Seria útil criar corpora anotados de problemas x linguagens.
- **Benchmarks:** Em codificação, conjuntos como *Sphere Engine* (antigo UVa, SPOJ) podem ser repurposed (diversas linguagens de soluções). Mas falta um benchmark dedicado para recomendação de linguagem.
- **Implementações e Ferramentas:** Os estudos citados não indicam frameworks abertos. O código de Twist et al. (2025) está disponível (GitHub) para avaliar LLMs ¹¹. Não há biblioteca padrão de referência.
- **Surveys Recentes:** Até onde achamos, não há survey focado exclusivamente nessa tarefa. O trabalho de Hossain et al. (2023) na *Software Practice & Experience* faz um levantamento de influências de traços na escolha de linguagem ¹⁰, e o estudo de Twist et al. (2025) pioneiro em LLMs serve como referência recente.
- **Outros Recursos:** Ferramentas de análise de código (por exemplo, LLM-based code assistants) poderão se tornar de fato recomendadores de linguagem. Livros textos de engenharia de software (p.e. *Somerville, 2016* citado em [131]) também discutem trade-offs de linguagem, podendo ser fonte de regras de conhecimento.

Fontes: Estudos citados no texto (Odieta et al. 2017 ⁶; Sholihin & Hidayati 2024 ⁹; Twist et al. 2025 ¹¹ ¹³; Amin et al. 2024 ¹² etc.) ilustram técnicas, limitações e tendências no tema. Estas referências (e outras mencionadas) fundamentam as observações acima.

¹ save-sd.github.io

<https://save-sd.github.io/2018/accepted/fathalla/index.pdf>

² RecSys 2023 - Accepted Contributions - RecSys – RecSys

<https://recsys.acm.org/recsys23/accepted-contributions/>

³ EARL Workshop

<https://earl-workshop.github.io/>

⁴ RecSys2025 — Workshops - RecSys

<https://recsys.acm.org/recsys25/workshops/>

⁵ ⁷ ⁸ Microsoft Word - RELATÓRIO REUNIÃO - QUALIS EVENTOS - COMPUTAÇÃO - sem CPF.doc

[https://www.gov.br/capes/pt-br/centrais-de-conteudo/documentos/avaliacao/](https://www.gov.br/capes/pt-br/centrais-de-conteudo/documentos/avaliacao/09012022_RELATORIOQUALISEVENTOS20172020COMPUTACAO.PDF)

[09012022_RELATORIOQUALISEVENTOS20172020COMPUTACAO.PDF](https://www.gov.br/capes/pt-br/centrais-de-conteudo/documentos/avaliacao/09012022_RELATORIOQUALISEVENTOS20172020COMPUTACAO.PDF)

⁶ Programming expertise graph. | Download Scientific Diagram

https://www.researchgate.net/figure/Programming-expertise-graph_fig8_318328095

⁹ A Forward Chaining Expert System for Personalized Programming Language Selection - Institut Teknologi Sepuluh Nopember (ITS), Indonesia

<https://scholar.its.ac.id/en/publications/a-forward-chaining-expert-system-for-personalized-programming-lan>

¹⁰ Analysis of Software Developers' Programming Language ...

<https://onlinelibrary.wiley.com/doi/full/10.1002/spe.3381>

¹¹ ¹³ LLMs Love Python: A Study of LLMs' Bias for Programming Languages and Libraries

<https://arxiv.org/html/2503.17181v1>

¹² Md Faizul AMIN | Doctoral Student | University of Aizu, Fukushima | Department of Computer and Information Systems | Research profile
<https://www.researchgate.net/profile/Md-Faizul-Amin>